

STAT40730 Final

James McClatchie

Table of contents

0.1	Packages used	2
1	Introduction	2
2	Part 1 – Analysis of ATP Challenger & Qualifying Matches (Surface Effects)	3
2.1	Research question and data description	3
2.1.1	Packages	3
2.1.2	Read the ATP 2022 Dataset	3
2.1.3	Overview of dataset	4
2.2	Data cleaning and derived variables	5
2.3	Numerical summaries by surface (using purrr)	6
2.4	Overall distribution of match length	8
2.5	Match length by surface	8
2.6	Serve performance by surface	10
2.6.1	Total aces per match by surface	10
2.6.2	Aces per serve point by surface	10
2.6.3	Double faults per match by surface	12
2.7	Rounds and Surface	12
2.8	Summary of findings for part 1	14
3	Part 2 – R Package: <i>janitor</i>	15
3.1	Purpose of package	15
3.2	Loading the package and data	15
3.2.1	Function 1 <code>clean_names()</code>	16
3.3	Function 2 <code>tabyl()</code>	17
3.4	Function 3 <code>get_dupes()</code>	19
3.4.1	Summary of Part 2	20
4	Part 3. Functions/Programming (S3)	21
4.1	Function: <code>surface_effect()</code>	21

4.2	S3 Methods	23
4.3	Worked examples	24
4.3.1	Interpretation of results	28
4.4	Summary of part 3	28
4.5	Conclusion	28
4.6	References	29

0.1 Packages used

```
library(tidyverse)
library(janitor)
```

1 Introduction

The main analytical component is presented in Part 1, where I examine ATP Challenger and qualifying matches played in 2022 using the ATP 2022 dataset. The central research question is: how does court surface (Clay, Hard, Grass, and Carpet) relate to match length and serve performance in these matches? To address this, I read in the raw data, applied a series of data cleaning steps, and created derived match-level variables such as `total_aces` and `total_df`. I then produced numerical summaries and visual comparisons of match duration and serve performance across surfaces.

In Part 2, I demonstrate the use of the R package `janitor`, which provides tools for examining and cleaning rectangular data frames. I apply key functions from the package, including `clean_names()`, `tabyl()`, and `get_dupes()`, to support common data cleaning and exploratory tasks such as standardising column names and generating frequency tables for surface by tournament round.

The final section, Part 3, focuses on functions and programming using S3. In this part, I develop a custom R function, `surface_effect()`, designed to compare a numeric outcome across groups, with court surface as the default grouping variable. The function returns an S3 object of class `surface_effect` and includes custom `print`, `summary`, and `plot` methods, providing a reusable framework for analysis, descriptive statistics, and visualisation.

2 Part 1 – Analysis of ATP Challenger & Qualifying Matches (Surface Effects)

2.1 Research question and data description

In this part, I analyse the **ATP Challenger and qualifying matches played in 2022**, using the file `atp_matches_qual_chall_2022.csv` stored in the same directory as this Quarto document.

The main research question is:

How does the court surface (Clay, Hard, Grass, Carpet) relate to match length and serve performance in ATP Challenger and qualifying matches in 2022?

The dataset contains one row per match and includes tournament information, player information, and detailed serve statistics for both the winner and the loser.

Key variables used here:

```
**Categorical variables**
  `surface` - court surface (Clay, Hard, Grass, Carpet)
  `round` - round of the tournament (e.g. R32, QF, SF, F)
  `tourney_level` - tournament level (here mostly Challenger/qualifying)
**Numeric variables**
  `minutes` - match duration in minutes
  `w_ace`, `l_ace` - aces served by winner and loser
  `w_df`, `l_df` - double faults by winner and loser
  `w_svpt`, `l_svpt` - total serve points played by winner and loser
```

These variables give us a mix of categorical and numerical data, satisfying the project guidelines.

2.1.1 Packages

```
library(tidyverse)
```

2.1.2 Read the ATP 2022 Dataset

```
matches_raw = read_csv("atp_matches_qual_chall_2022.csv")
```

Rows: 10016 Columns: 49

-- Column specification -----

Delimiter: ","

chr (14): tourney_id, tourney_name, surface, tourney_level, winner_entry, wi...

dbl (35): draw_size, tourney_date, match_num, winner_id, winner_seed, winner...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

2.1.3 Overview of dataset

```
glimpse(matches_raw)
```

Rows: 10,016

Columns: 49

\$ tourney_id	<chr> "2022-0059", "2022-0059", "2022-0059", "2022-0059", ~
\$ tourney_name	<chr> "Salinas CH", "Salinas CH", "Salinas CH", "Salinas ~
\$ surface	<chr> "Hard", "Hard", "Hard", "Hard", "Hard", "Hard", "Ha~
\$ draw_size	<dbl> 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, ~
\$ tourney_level	<chr> "C", "C", "C", "C", "C", "C", "C", "C", "C", "C", "C", "~
\$ tourney_date	<dbl> 20220404, 20220404, 20220404, 20220404, 20220404, 2~
\$ match_num	<dbl> 238, 239, 240, 241, 242, 243, 244, 245, 249, 250, 2~
\$ winner_id	<dbl> 111569, 104907, 111788, 138485, 200587, 105031, 106~
\$ winner_seed	<dbl> 6, NA, 5, 10, 4, 11, 3, NA, 8, 5, 4, 3, NA, 1, 2, N~
\$ winner_entry	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ winner_name	<chr> "Gage Brymer", "Jose Rubin Statham", "Naoki Nakagaw~
\$ winner_hand	<chr> "R", "R", "R", "R", "R", "R", "R", "R", "R", "R", "R", "~
\$ winner_ht	<dbl> 185, 180, 180, 188, 185, 193, 191, 191, 188, 180, 1~
\$ winner_ioc	<chr> "USA", "NZL", "JPN", "PER", "USA", "IND", "ARG", "G~
\$ winner_age	<dbl> 27.1, 34.9, 25.3, 26.2, 23.9, 34.4, 27.5, 20.5, 25.~
\$ loser_id	<dbl> 105419, 111806, 105233, 122635, 210300, 144747, 106~
\$ loser_seed	<dbl> NA, 9, NA, NA, NA, NA, NA, NA, 12, 6, NA, 10, 11, 2, 7, ~
\$ loser_entry	<chr> "WC", NA, "Alt", "Alt", "WC", "Alt", NA, NA, NA, NA, ~
\$ loser_name	<chr> "Patricio Alvarado", "Fajing Sun", "Jeevan Nedunche~
\$ loser_hand	<chr> "R", "R", "L", "L", "U", "R", "R", "R", "R", "R", "R", "~
\$ loser_ht	<dbl> 175, 178, 173, 183, NA, 193, 3, 188, 185, 180, 188, ~
\$ loser_ioc	<chr> "ECU", "CHN", "IND", "PER", "ECU", "ZIM", "PER", "M~

```

$ loser_age      <dbl> 32.4, 25.5, 33.4, 29.5, 14.7, 25.4, 26.7, 26.9, 27.~
$ score          <chr> "6-2 6-1", "5-7 7-5 6-2", "6-4 6-3", "6-1 6-1", "6--~
$ best_of        <dbl> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, ~
$ round          <chr> "Q1", "Q1", "Q1", "Q1", "Q1", "Q1", "Q1", "Q1", "Q2~
$ minutes        <dbl> 77, 161, 77, 51, 47, 49, 145, 145, 95, 87, 158, 120~
$ w_ace          <dbl> 0, 7, 5, 4, 3, 8, 4, 1, 13, 6, 7, 1, NA, 4, 12, 2, ~
$ w_df           <dbl> 3, 3, 3, 0, 0, 2, 3, 6, 5, 3, 1, 1, NA, 2, 3, 3, 3,~
$ w_svpt         <dbl> 38, 92, 52, 34, 27, 35, 85, 90, 66, 53, 92, 88, NA,~
$ w_1stIn        <dbl> 23, 64, 27, 22, 17, 22, 59, 49, 36, 23, 61, 63, NA,~
$ w_1stWon       <dbl> 18, 48, 22, 19, 15, 18, 44, 31, 29, 21, 40, 45, NA,~
$ w_2ndWon       <dbl> 9, 11, 17, 9, 9, 10, 13, 18, 14, 21, 19, 16, NA, 13~
$ w_SvGms        <dbl> 7, 16, 9, 7, 6, 7, 13, 12, 10, 10, 14, 12, NA, 9, 1~
$ w_bpSaved      <dbl> 2, 2, 1, 0, 0, 0, 3, 8, 5, 1, 1, 4, NA, 0, 1, 7, 3,~
$ w_bpFaced      <dbl> 3, 5, 1, 0, 0, 0, 4, 12, 6, 1, 4, 5, NA, 0, 1, 10, ~
$ l_ace          <dbl> 0, 2, 4, 1, 0, 0, 7, 4, 1, 5, 5, 7, NA, 6, 10, 2, 1~
$ l_df           <dbl> 3, 3, 3, 4, 0, 3, 1, 7, 3, 2, 1, 1, NA, 1, 2, 1, 1,~
$ l_svpt         <dbl> 61, 110, 72, 42, 37, 42, 80, 90, 58, 58, 98, 74, NA~
$ l_1stIn        <dbl> 33, 76, 49, 29, 19, 24, 40, 59, 36, 37, 64, 46, NA,~
$ l_1stWon       <dbl> 15, 48, 29, 11, 4, 9, 33, 37, 19, 26, 35, 37, NA, 2~
$ l_2ndWon       <dbl> 11, 17, 12, 6, 6, 8, 15, 11, 14, 10, 20, 15, NA, 8,~
$ l_SvGms        <dbl> 8, 16, 10, 7, 7, 7, 13, 12, 10, 10, 13, 12, NA, 9, ~
$ l_bpSaved      <dbl> 8, 4, 6, 2, 3, 4, 2, 9, 4, 2, 7, 1, NA, 7, 4, 2, 4,~
$ l_bpFaced      <dbl> 14, 9, 9, 7, 9, 9, 5, 14, 7, 4, 12, 2, NA, 10, 8, 7~
$ winner_rank    <dbl> 537, 725, 516, 613, 496, 638, 489, 830, 580, 516, 4~
$ winner_rank_points <dbl> 57, 28, 61, 43, 66, 39, 68, 19, 49, 61, 66, 68, 19,~
$ loser_rank     <dbl> NA, 629, NA, 1306, NA, 2129, 702, 650, 537, 725, 61~
$ loser_rank_points <dbl> NA, 40, NA, 4, NA, 1, 31, 37, 57, 28, 43, 39, 75, 7~

```

This glimpse shows that the data frame has one row per match and includes tournament level variables (e.g. `tourney_name`, `surface`), player characteristics, and detailed serve statistics for the winner and the loser.

2.2 Data cleaning and derived variables

For the surface analysis, I focus on the following:

Keep only matches where:

```
`surface` is not missing.
```

```
`minutes` is positive (to remove any erroneous 0 minute matches).
```

Create total serve statistics per match:

```
`total_aces = w_ace + l_ace`
```

```
`total_df    = w_df + l_df`
`total_svpt  = w_svpt + l_svpt`
```

I also keep a smaller subset of relevant columns to simplify later code.

```
#|label: clean data

#only keeping rows with known surface + positive match duration
matches = matches_raw |>
  #total serve stats for match
  dplyr::filter(!is.na(surface), minutes > 0) |> dplyr::mutate(
    total_aces = w_ace + l_ace,
    total_df = w_df + l_df,
    total_svpt = w_svpt + l_svpt
  )
#only variables needed
) |> dplyr::select(
  tourney_name, surface, tourney_level, round, minutes,
  w_ace, l_ace, w_df, w_svpt, l_svpt, total_aces, total_df,
  total_svpt
)

#check surfaces and their frequencies after clean

matches |> dplyr::count(surface, sort = TRUE)
```

```
# A tibble: 4 x 2
  surface      n
  <chr>   <int>
1 Clay    4907
2 Hard    4369
3 Grass    343
4 Carpet    49
```

This shows how many matches were played on each surface in the 2022 Challenger/qualifying season. Clay and Hard dominate, with fewer matches on Grass and Carpet.

2.3 Numerical summaries by surface (using purrr)

Here I summarise the main numeric variables by surface, using `purrr::map_dfr()` to avoid repeating code.

variables of interest: `minutes total_aces total_df total_svpt`

```

#vector of numeric variables
numeric_vars = c("minutes","total_aces","total_df","total_svpt")

#function to summarise one variable by surface

summarise_by_surface = function(var) {
matches |>
dplyr::group_by(surface) |>
dplyr::summarise(
variable = var,
n = dplyr::n(),
mean = mean(.data[[var]], na.rm = TRUE),
sd = sd(.data[[var]], na.rm = TRUE),
min = min(.data[[var]], na.rm = TRUE),
max = max(.data[[var]], na.rm = TRUE),
.groups = "drop"
)
}

#purrr::map_dfr()
surface_summary = numeric_vars |>
  purrr::map_dfr(summarise_by_surface)

surface_summary

```

```

# A tibble: 16 x 7
  surface variable      n  mean   sd  min  max
  <chr>   <chr>   <int> <dbl> <dbl> <dbl> <dbl>
1 Carpet minutes     49  89.6  24.2   36  141
2 Clay   minutes  4907 104.   34.8    3  260
3 Grass  minutes   343  96.6  36.1   22  220
4 Hard   minutes  4369 100.   32.5    6  217
5 Carpet total_aces    49  21.1   9.56    4   50
6 Clay   total_aces  4907   5.74   4.52    0   32
7 Grass  total_aces   343  12.1   6.75    0   42
8 Hard   total_aces  4369  10.4   6.19    0   53
9 Carpet total_df     49   6.29   3.69    1   18
10 Clay  total_df   4907   5.85   3.63    0   28
11 Grass total_df   343   5.77   3.21    0   16
12 Hard  total_df   4369   5.88   3.43    0   28
13 Carpet total_svpt    49 145.   38.0   56  224
14 Clay  total_svpt  4907 141.   42.2    4  288

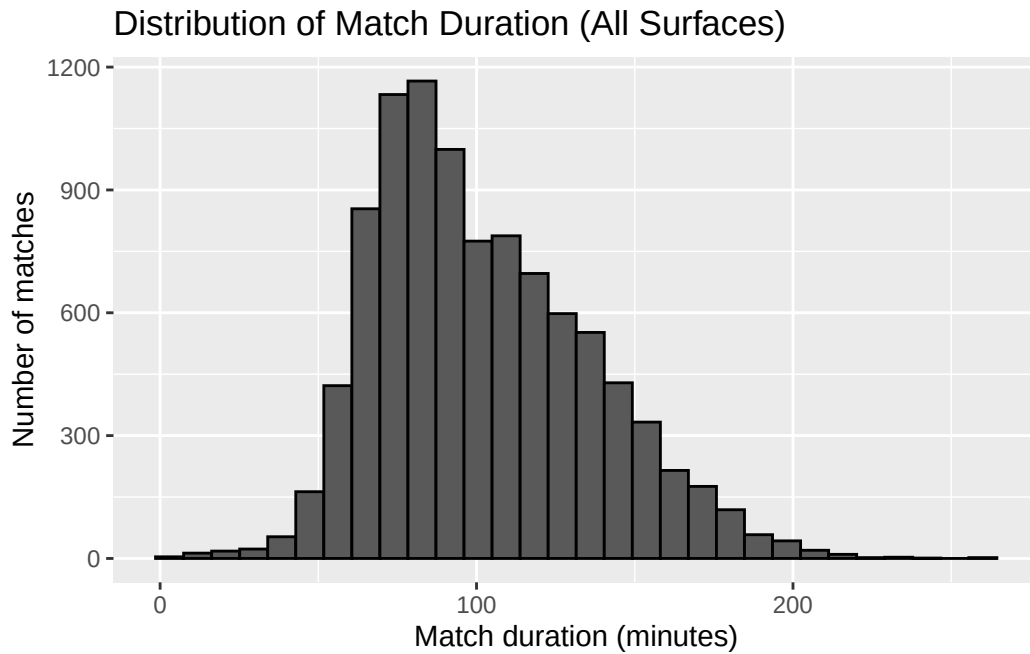
```

15	Grass	total_svpt	343	149.	49.5	23	327
16	Hard	total_svpt	4369	143.	41.4	4	307

This table uses `map_dfr()` from *purrr* to generate summaries of several numeric variables at once. For each surface, we see the mean, standard deviation, minimum and maximum of match duration, total aces, double faults and serve points.

2.4 Overall distribution of match length

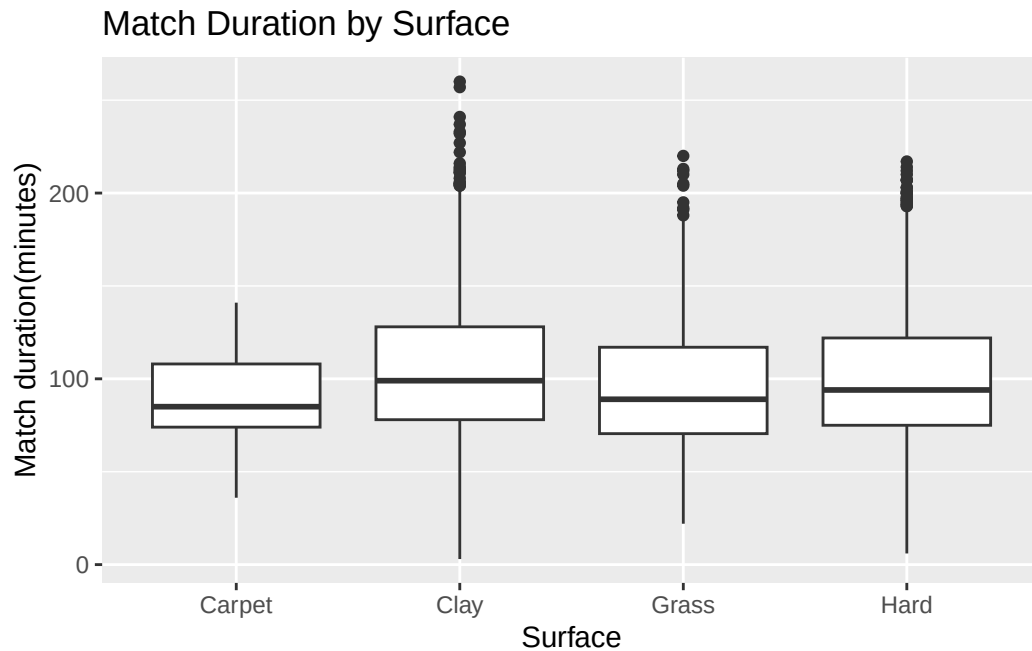
Before comparing surfaces, it is useful to look at the overall distribution of match duration.



Most matches last between 60 and 140 minutes, with a long tail towards longer matches. Very short matches are relatively rare after removing the 0 minute entries.

2.5 Match length by surface

Next is the comparison I made on how long matches last on different surfaces.



This boxplot shows the distribution of match duration for each surface. We can visually compare medians and the spread of durations on Clay, Hard, Grass, and Carpet.

In addition to the boxplot, I will compute some numeric summaries of minutes by surface

```
#|label: minutes by surface
minutes_summary = matches |>
  group_by(surface) |>
  summarise(
    n_matches = n(),
    mean_minutes = mean(minutes, na.rm = TRUE),
    median_minutes = median(minutes, na.rm = TRUE),
    sd_minutes = sd(minutes, na.rm = TRUE),
    .groups = "drop"
  )
```

minutes_summary

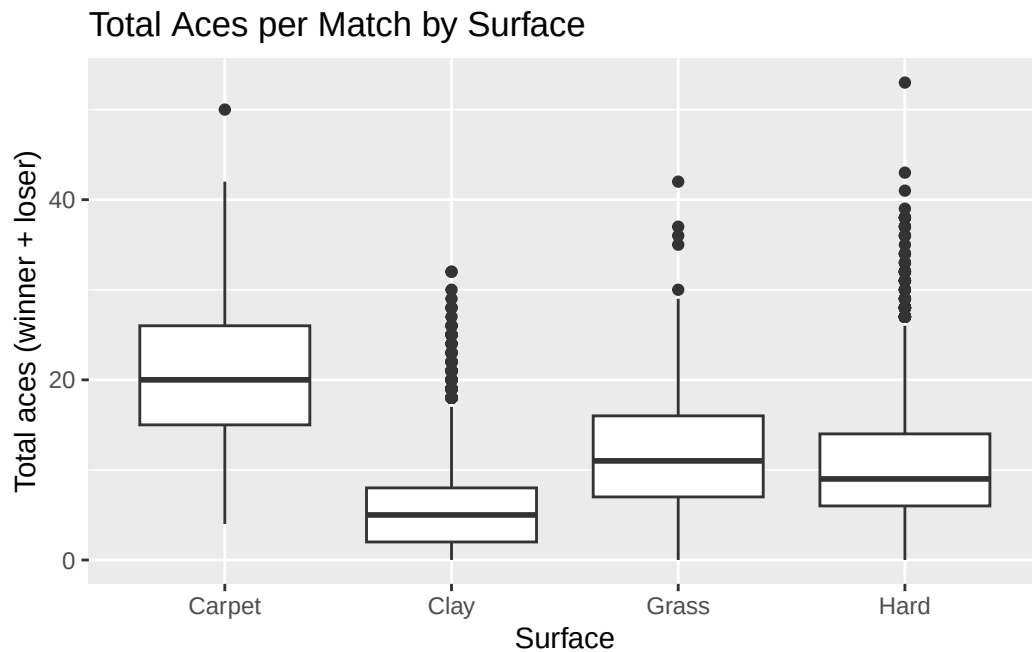
```
# A tibble: 4 x 5
  surface n_matches mean_minutes median_minutes sd_minutes
  <chr>     <int>      <dbl>         <dbl>      <dbl>
1 Carpet      49      89.6           85        24.2
2 Clay    4907     104.           99        34.8
```

3	Grass	343	96.6	89	36.1
4	Hard	4369	100.	94	32.5

Clay matches lasted the longest on average, followed by Hard, then Grass, with Carpet producing the shortest matches which is consistent with known differences in court speed.

2.6 Serve performance by surface

2.6.1 Total aces per match by surface



Total aces vary by surface, with Carpet and Grass producing more aces, Hard next and Clay generating the fewest. It could be interpreted then that surface speed influences serve dominance.

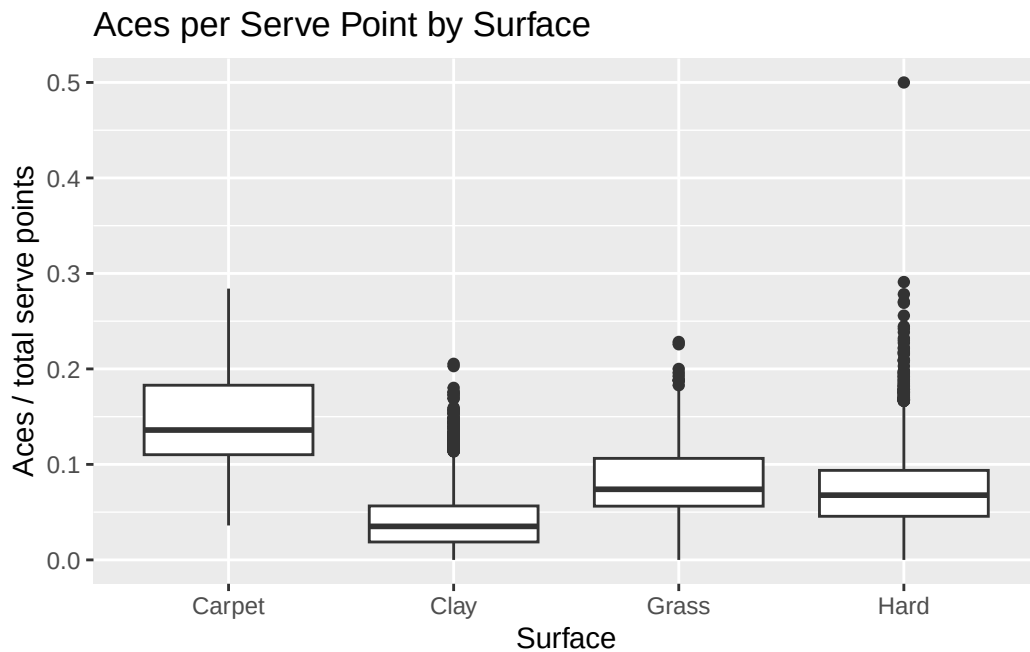
2.6.2 Aces per serve point by surface

```
# A tibble: 4 x 4
  surface n_matches mean_ace_rate median_aces_rate
  <chr>     <int>         <dbl>         <dbl>
1 Carpet      49         0.145         0.136
2 Clay     4907         0.0407         0.0351
3 Grass     343         0.0826         0.0739
```

4 Hard 4369 0.0730 0.0678

Aces per serve point rates reflect the surface/speed hierarchy, with Carpet producing the highest proportional ace rates (skewed because of difference in total matches played on each surface type), followed by Grass and Hard, while Clay shows the lowest serve dominance.

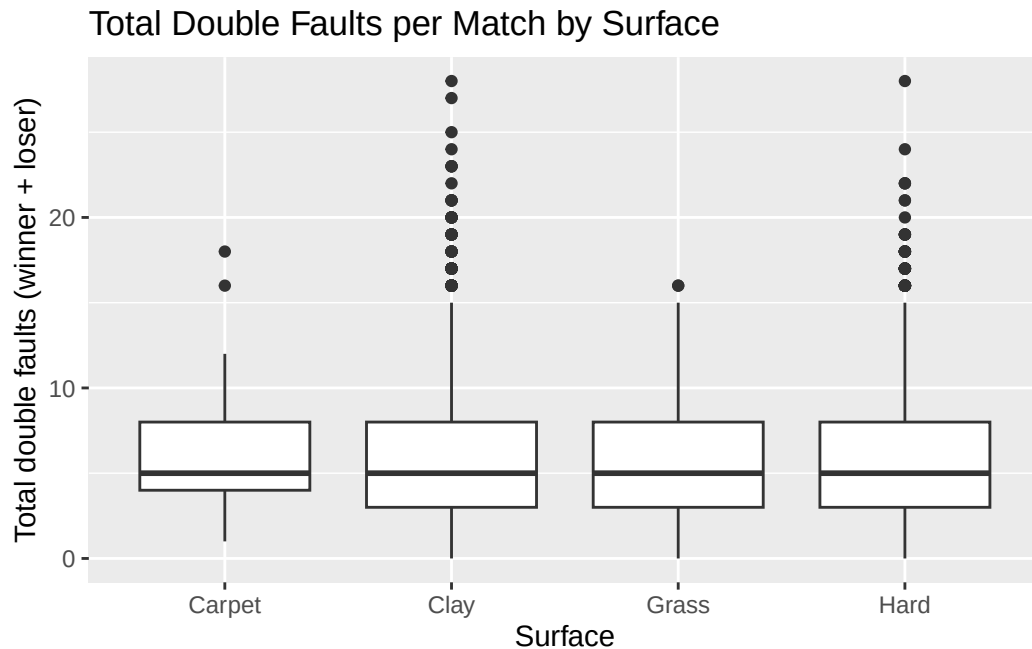
```
matches |>
  dplyr::filter(!is.na(aces_per_svpt)) |>
  ggplot2::ggplot(ggplot2::aes(x = surface, y = aces_per_svpt)) +
  ggplot2::geom_boxplot() +
  ggplot2::labs(
    title = "Aces per Serve Point by Surface",
    x = "Surface",
    y = "Aces / total serve points"
  )
```



As mentioned, carpet produces the highest serving effectiveness, followed by Grass and Hard, while Clay has the lowest proportion of serve points ending in aces. Because this statistic standardises for the number of serve points played, it gives a comparison across matches and highlights how faster surfaces reward powerful serving.

2.6.3 Double faults per match by surface

Warning: Removed 53 rows containing non-finite outside the scale range (``stat_boxplot()``).



Total double faults show no strong differences across surfaces. All four have similar medians and spread, suggesting that double fault rates are not heavily influenced by court speed and instead reflect player specific serving consistency.

2.7 Rounds and Surface

As an extra categorical view, I will check how matches by tournament round are distributed across surfaces

```
# A tibble: 31 x 3
  surface round n_matches
  <chr>   <chr>     <int>
1 Carpet F         1
2 Carpet Q1        12
3 Carpet Q2         6
4 Carpet QF         4
5 Carpet R16        8
```

```

6 Carpet R32          16
7 Carpet SF           2
8 Clay F           101
9 Clay Q1          1127
10 Clay Q2          672
# i 21 more rows

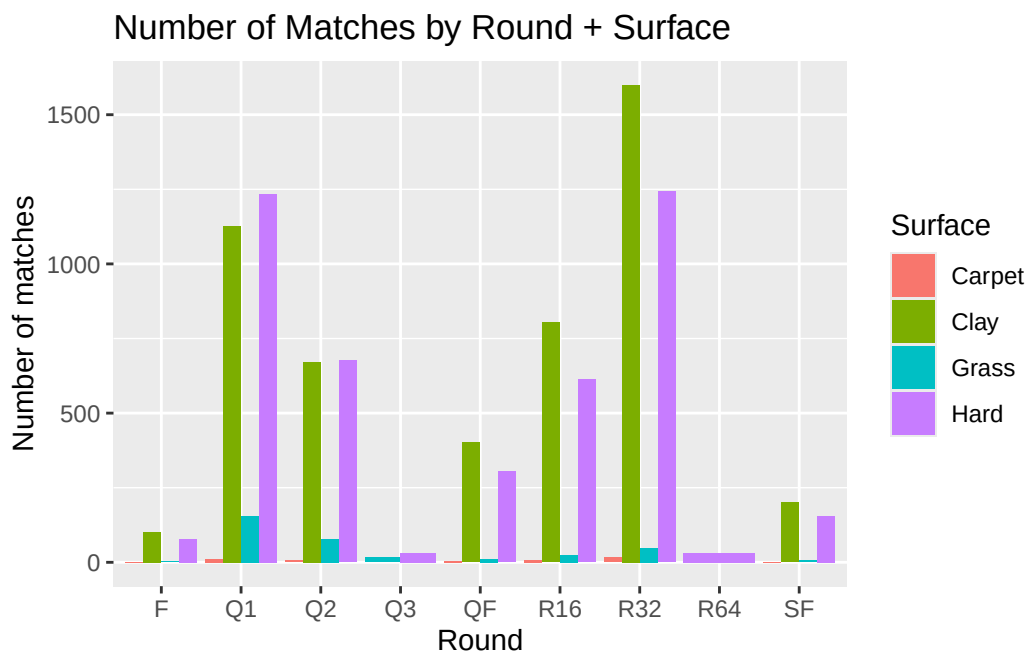
```

The round surface counts show that Clay and Hard courts dominate the dataset across all rounds, while Grass and especially Carpet have far fewer matches. This imbalance explains why some surface comparisons appear more variable and should be interpreted with caution.

```

ggplot(round_surface_counts,
       aes(x = round, y = n_matches,
           fill = surface)) +
geom_col(position = "dodge") +
labs(title = "Number of Matches by Round + Surface",
     x = "Round",
     y = "Number of matches",
     fill = "Surface")

```



The round and surface distribution shows that Clay and Hard dominate all stages of the tournament calendar, while Grass and Carpet have far fewer matches. This is important context for interpreting surface differences elsewhere in the analysis.

2.8 Summary of findings for part 1

This exploratory analysis examined how court surface relates to match characteristics and serve performance in ATP Challenger and qualifying tournaments during 2022. Several patterns emerge:

I. Strong surface imbalance in the dataset.

Clay and Hard dominate the calendar, accounting for the overwhelming majority of matches across all rounds. Grass tournaments are limited, and Carpet events are extremely rare. This uneven distribution affects the stability of comparisons, with Clay and Hard giving the most reliable patterns.

II. Match duration differs meaningfully across surfaces.

Clay matches are consistently the longest, followed by Hard and then Grass, with Carpet producing the shortest matches. This aligns with known court speed characteristics: slower surfaces (Clay) prolong rallies, while faster courts (Grass/Carpet) shorten point length and match duration (Nag, 2022).

III. Serve performance varies strongly with surface speed.

Total aces: Carpet and Grass produce the most aces, Hard is moderate, and Clay shows clearly the lowest ace counts.

Aces per serve point: Standardising by serve points confirms the same hierarchy - Carpet has the highest proportional ace rate, followed by Grass and Hard, with Clay far lower. Faster surfaces reward unreturnable serving and flatter ball trajectories (Vaverka et al., 2018).

IV. Double fault behaviour is relatively surface independent.

While aces show large surface differences, double faults remain broadly similar across surfaces. Medians and spreads overlap heavily, suggesting that serving consistency (avoiding double faults) depends more on player skill and match context than on court type.

V. Round by surface distributions provide essential context.

Early rounds (Q1, Q2, R32) dominate the dataset, especially on Clay and Hard, whereas Grass and Carpet appear only sporadically. These structural differences help explain why some surface comparisons appear more variable or contain more outliers.

Literature Support

These results are consistent with published research on how court surfaces influence match characteristics. For example, (Revan, Doğan and Arıkan, 2021) analysed Grand Slam matches and reported that faster surfaces produce more aces and shorter matches, while clay results in fewer aces and longer rallies due to its slower playing conditions. They also found that double faults vary much less across surfaces, a pattern that appears strongly in this Challenger dataset as well. The alignment between my findings and those reported in the literature suggests that the surface effects observed here reflect broader, established differences in how tennis is played on Clay, Hard, Grass, and Carpet courts.

Overall conclusion

Surface type is clearly associated with both match duration and serve dominance. Faster surfaces amplify the impact of strong serving which leads to shorter matches, more aces, and higher ace per serve point rates, while slower surfaces reduce serve effectiveness and lengthen matches. However, double faults appear largely unaffected by surface. These findings broadly mirror well known physical and tactical differences between tennis surfaces and demonstrate how match level statistics reflect underlying playing conditions.

3 Part 2 – R Package: *janitor*

3.1 Purpose of package

This section I will demonstrate the use of the R package *janitor* on the ATP Challenger and qualifying tennis dataset from part 1

The *janitor* package provides tools for examining and cleaning data in rectangular dataframes. It's designed to help with common data cleaning tasks like:

- standardising column names
- producing quick frequency tables and crosstabs
- detecting duplicate records in a dataset

The package was developed by Sam Firke (author and maintainer) with contributions from Bill Denney, Chris Haid, Ryan Knight, Malte Grosser, and Jonathan Zadra (Firke et al., 2024). According to CRAN description, *janitor* “has simple functions for examining and cleaning dirty data” and is intended for beginner to intermediate R users (like myself), while still being convenient for more advanced users (Firke et al., 2024).

I will be using the three main functions:

`clean_names()` standardises column names in a data frame.

`tabyl()` creates frequency tables and crosstabs.

`get_dupes()` finds duplicate rows based on specified columns.

3.2 Loading the package and data

```
library(tidyverse)
library(janitor) #new package for prt 2

#read atp dataset used in prt 1 again
matches_raw = read_csv("atp_matches_qual_chall_2022.csv", show_col_types = FALSE)
```

Using the same dataset, now looking at how *janitor* can help organise and explore the data more efficiently.

3.2.1 Function 1 `clean_names()`

The raw dataset has a mix of naming style(lowercase, underscores, sometimes less descriptive). The `clean_names()` function creates consistent, machine friendly column names (e.g. converting spaces or punctuation to underscores, making everything lowercase)

```
#|label: janitor clean names

matches_clean = matches_raw |>
janitor::clean_names()

#original vs cleaned
tibble(
  original = names(matches_raw)[1:10],
  cleaned = names(matches_clean)[1:10]
)
```

```
# A tibble: 10 x 2
  original      cleaned
  <chr>         <chr>
1 tourney_id    tourney_id
2 tourney_name  tourney_name
3 surface       surface
4 draw_size     draw_size
5 tourney_level tourney_level
6 tourney_date  tourney_date
7 match_num     match_num
8 winner_id     winner_id
9 winner_seed   winner_seed
10 winner_entry winner_entry
```

Because the ATP dataset already uses consistent snake_case column names, `clean_names()` does not modify them. Also shows that the function preserves existing clean names and only makes changes when needed.

Here is an example of `janitor` in use

```
demo_df = tibble(
  "Match Number" = 1:3,
  "Winner.ID" = c("A", "B", "C"),
  "Surface Type" = c("Clay", "Hard", "Grass")
)

janitor::clean_names(demo_df)
```

```
# A tibble: 3 x 3
  match_number winner_id surface_type
      <int>   <chr>      <chr>
1         1     A      Clay
2         2     B      Hard
3         3     C      Grass
```

3.3 Function 2 `tabyl()`

The `tabyl()` function provides a convenient way to compute frequency tables and cross tabulations, similar to Excel.

3.3.0.1 One way frequency table: number of matches per surface

```
#|label: janitor tabylt surface

surface_counts = matches_clean |>
  janitor::tabyl(surface)

surface_counts
```

surface	n	percent
Carpet	49	0.004892173
Clay	5125	0.511681310
Grass	343	0.034245208
Hard	4499	0.449181310

This table shows the number of matches played on each surface in the dataset and `tabyl()` returns a frequency table with both counts and `percent`.

To see proportions clearly, I can add a percentage column

```
#|label: janitor tabyl surface percent

surface_counts |>
  janitor::adorn_totals("row") |>
  janitor::adorn_percentages("col") |>
  janitor::adorn_pct_formatting(digits = 1)
```

surface	n	percent
Carpet	0.0	0.5%
Clay	0.5	51.2%
Grass	0.0	3.4%
Hard	0.4	44.9%
Total	1.0	100.0%

I'm using the helper functions `adorn_totals()`, `adorn_percentages()`, and `adorn_pct_formatting()` together with `tabyl()`. They add a total row and covert counts into readable percentages. This shows that Clay and Hard courts are the majority, as seen in part 1, while Grass and Carpet appear much less frequently.

3.3.0.2 Two way table: surface by round

```
#|label: janitor tabyl surface round

surface_round_table = matches_clean |>
  janitor::tabyl(surface, round)

surface_round_table
```

surface	F	Q1	Q2	Q3	QF	R16	R32	R64	SF
Carpet	1	12	6	0	4	8	16	0	2
Clay	101	1268	717	16	404	808	1609	0	202
Grass	3	156	78	16	12	24	48	0	6
Hard	78	1310	692	32	312	628	1259	32	156

This table summarises how matches are distributed across combinations of court surface and tournament round. Each cell showing the number of matches played on a given surface in a specific round. Again, Clay and Hard dominate the early qualifying rounds (Q1,Q2 etc.) as well as R32. This function provides a quick and effective way to understand categorical relationships in the data without additional reshaping.

Again I can show percentages, for each surface, to see how matches are spread by round.

```
#|label: janitor tabyl surface round percent
```

```
surface_round_table |>
  janitor::adorn_totals("row")|>
  janitor::adorn_percentages("row")|>
  janitor::adorn_pct_formatting(digits = 1)
```

surface	F	Q1	Q2	Q3	QF	R16	R32	R64	SF
Carpet	2.0%	24.5%	12.2%	0.0%	8.2%	16.3%	32.7%	0.0%	4.1%
Clay	2.0%	24.7%	14.0%	0.3%	7.9%	15.8%	31.4%	0.0%	3.9%
Grass	0.9%	45.5%	22.7%	4.7%	3.5%	7.0%	14.0%	0.0%	1.7%
Hard	1.7%	29.1%	15.4%	0.7%	6.9%	14.0%	28.0%	0.7%	3.5%
Total	1.8%	27.4%	14.9%	0.6%	7.3%	14.7%	29.3%	0.3%	3.7%

3.4 Function 3 get_dupes()

Finding potential duplicate records

```
#|label: janitor gets duplicates!
```

```
matches_clean |>
  filter(!is.na(minutes)) |>
  janitor::get_dupes(minutes)
```

```
# A tibble: 9,680 x 50
```

	minutes	dupe_count	tourney_id	tourney_name	surface	draw_size	tourney_level
	<dbl>	<int>	<chr>	<chr>	<chr>	<dbl>	<chr>
1	80	149	2022-0059	Salinas CH	Hard	32	C
2	80	149	2022-0213	San Luis Potos~	Clay	32	C
3	80	149	2022-0221	Tampere CH	Clay	32	C
4	80	149	2022-0315	Newport	Grass	32	A
5	80	149	2022-0341	Metz	Hard	32	A
6	80	149	2022-0363	Knoxville CH	Hard	32	C
7	80	149	2022-0375	Montpellier	Hard	32	A

```

      8      80      149 2022-0407 Rotterdam      Hard      32 A
      9      80      149 2022-0414 Hamburg      Clay      32 A
     10      80      149 2022-0425 Barcelona      Clay      64 A
# i 9,670 more rows
# i 43 more variables: tourney_date <dbl>, match_num <dbl>, winner_id <dbl>,
#   winner_seed <dbl>, winner_entry <chr>, winner_name <chr>,
#   winner_hand <chr>, winner_ht <dbl>, winner_ioc <chr>, winner_age <dbl>,
#   loser_id <dbl>, loser_seed <dbl>, loser_entry <chr>, loser_name <chr>,
#   loser_hand <chr>, loser_ht <dbl>, loser_ioc <chr>, loser_age <dbl>,
#   score <chr>, best_of <dbl>, round <chr>, w_ace <dbl>, w_df <dbl>, ...

```

When I apply `get_dupes()` only to the minutes variable (after removing NAs), I find that some match duration occur many times across the season. For example, the output shows that matches lasting exactly 80 minutes appear 149 times in the dataset, spread across a wide range of tournaments (e.g. Salinas CH, Montpellier, Rotterdam, Hamburg, Barcelona). The `dupe_count` column indicates the total number of matches sharing that duration, repeated on each row of the group. This illustrates how `get_dupes()` can highlight common match lengths and identify heavily repeated values in a numeric variable, not just duplicated records in an ID field.

3.4.1 Summary of Part 2

In this section, I demonstrated how the `janitor` package can support efficient data cleaning and exploratory analysis using the same ATP Challenger and qualifying dataset from Part 1. Although `janitor` is often used for tidying messy data, its functions are also valuable for quickly inspecting the structure of already clean datasets such as this one.

The first function, `clean_names()`, standardised variable names into consistent `snake_case`. Because the ATP dataset already followed this format, no changes were made, reinforcing that `clean_names()` is non destructive and only modifies names when needed. A small illustrative example showed how the function behaves on messier column names.

The second function, `tabyl()`, provided flexible frequency tables and cross tabulations. A one way table revealed the strong surface imbalance in the dataset (Clay and Hard dominating match counts), while a two way table of surface $\times$ round highlighted structural differences in the tournament calendar, such as the concentration of Clay and Hard matches in the qualifying rounds. The `adorn_*` helper functions allowed counts to be converted into percentages and totals, making the tables more interpretable.

The third function, `get_dupes()`, identified repeated match durations after removing missing values. This showed that certain match lengths (e.g. 80 minutes) occur frequently across many tournaments, which reflects patterns in men's best of three play rather than data entry issues.

The function's grouping and `dupe_count` output provide a useful way to detect repeated values in numeric fields as well as categorical identifiers.

Overall, the `janitor` package offered clear and efficient tools for inspecting, cleaning, and summarising the structure of the dataset. While the ATP data was relatively tidy to begin with, `janitor` helped highlight underlying distributional patterns and provided fast, reproducible summaries that complement the more detailed statistical analysis performed in Part 1.

4 Part 3. Functions/Programming (S3)

```
#|label: part3 setup
#|message: false
#|warning: false

library(dplyr)
library(ggplot2)
```

4.1 Function: `surface_effect()`

A small analysis function to compare a numeric outcome across groups (default: tennis surface)

Returns an S3 object of class “`surface_effect`” with:

- cleaned data used in the analysis
- group wise summary statistics
- Kruskal Wallis test (robust to skew/outliers)

```
#|label: part3 surface effect function

surface_effect = function(data, response,
                           group = "surface",
                           drop_na = TRUE,
                           run_test = TRUE)
{#smome basic input checks
  if (!is.data.frame(data)) {stop("data must be a data.frame or tibble.")}

  if (!is.character(response) || length(response) != 1) {stop("response must
be a single character string
```

```

naming a numeric column.")}

  if (!is.character(group) || length(group) != 1) {stop("group must be a
single character string
naming a grouping column.")}

  if (!response %in% names(data)) {stop("response column not found in data: ", response)}

  if (!group %in% names(data)) {stop("group column not found in data: ", group)}

  if (!is.numeric(data[[response]])) {stop("response column must be numeric:
", response)}

#subset to relevant columns
df = data |>
dplyr::select(dplyr::all_of(c(group, response)))

#handle missing values
if (isTRUE(drop_na)) {df = df |>
dplyr::filter(
  !is.na(.data[[group]]),
  !is.na(.data[[response]])}

#group wise summary statistics
summary_tbl = df |>
dplyr::group_by(.data[[group]]) |>
dplyr::summarise(
  n      = dplyr::n(),
  mean   = mean(.data[[response]]),
  median = median(.data[[response]]),
  sd     = sd(.data[[response]]),
  min    = min(.data[[response]]),
  max    = max(.data[[response]]),
  .groups = "drop") |>
dplyr::rename(group_value = 1)

#Kruskal-Wallis test
# H0: distributions are the same across groups
test_out = NULL
if (isTRUE(run_test))
{if (dplyr::n_distinct(df[[group]]) >= 2) {test_out = stats::kruskal.test(
  df[[response]] ~ df[[group]])}}

```

```

#S3 object
out = list(
  call      = match.call(),
  response  = response,
  group     = group,
  data_used = df,
  summary_tbl = summary_tbl,
  test      = test_out)

class(out) = "surface_effect"
out
}

```

4.2 S3 Methods

```

#|label: S3-Methods

# S3 methods for class "surface_effect"
# - print.surface_effect(): quick overview (not the full stats table)
# - summary.surface_effect(): detailed stats + test output
# - plot.surface_effect(): visual comparison across groups

print.surface_effect <- function(x, ...) {
  n_total <- nrow(x$data_used)
  n_groups <- dplyr::n_distinct(x$data_used[[x$group]])
  grp_vals <- sort(unique(as.character(x$data_used[[x$group]])))

  cat("surface_effect object\n")
  cat("  Response:", x$response, "\n")
  cat("  Grouping variable:", x$group, "\n")
  cat("  Observations used:", n_total, "\n")
  cat("  Number of groups:", n_groups, "\n")
  cat("  Groups:", paste(grp_vals, collapse = ", "), "\n")

  invisible(x)
}

summary.surface_effect <- function(object, ...) {
  cat("Summary for surface_effect\n")
  cat("  Response:", object$response, "\n")

```

```

cat("  Grouping variable:", object$group, "\n\n")

cat("Group-wise descriptive statistics:\n")
print(object$summary_tbl)

if (!is.null(object$test)) {
  cat("\nKruskal-Wallis test (non-parametric):\n")
  cat("  H0: the distribution of", object$response,
"is the same across", object$group,
"\n")
cat("  statistic:", unname(object$test$statistic), "\n")
cat("  df:",
unname(object$test$parameter), "\n")
cat("  p-value:",
object$test$p.value, "\n")
} else {
cat("\nKruskal-Wallis test
was not run (or insufficient groups).\n")
}

invisible(object)
}

plot.surface_effect <- function(x, ...) {
  df <- x$data_used
  ggplot2::ggplot(df, ggplot2::aes(x = .data[[x$group]],
    y = .data[[x$response]])) +
    ggplot2::geom_boxplot(outlier.alpha = 0.3) +
    ggplot2::geom_jitter(width = 0.15, alpha = 0.2) +
    ggplot2::labs(title =
paste("Distribution of",
x$response, "by", x$group),
x = x$group,
y = x$response)
}

```

4.3 Worked examples

```

#|label: worked example
# Example 1: match duration by surface

```

```
se_minutes = surface_effect(
  data      = matches,
  response  = "minutes",
  group     = "surface",
  drop_na   = TRUE,
  run_test  = TRUE
)

# print() method (runs automatically)
se_minutes
```

```
surface_effect object
  Response: minutes
  Grouping variable: surface
  Observations used: 9668
  Number of groups: 4
  Groups: Carpet, Clay, Grass, Hard
```

```
# summary() method
summary(se_minutes)
```

Summary for surface_effect

```
Response: minutes
Grouping variable: surface
```

Group-wise descriptive statistics:

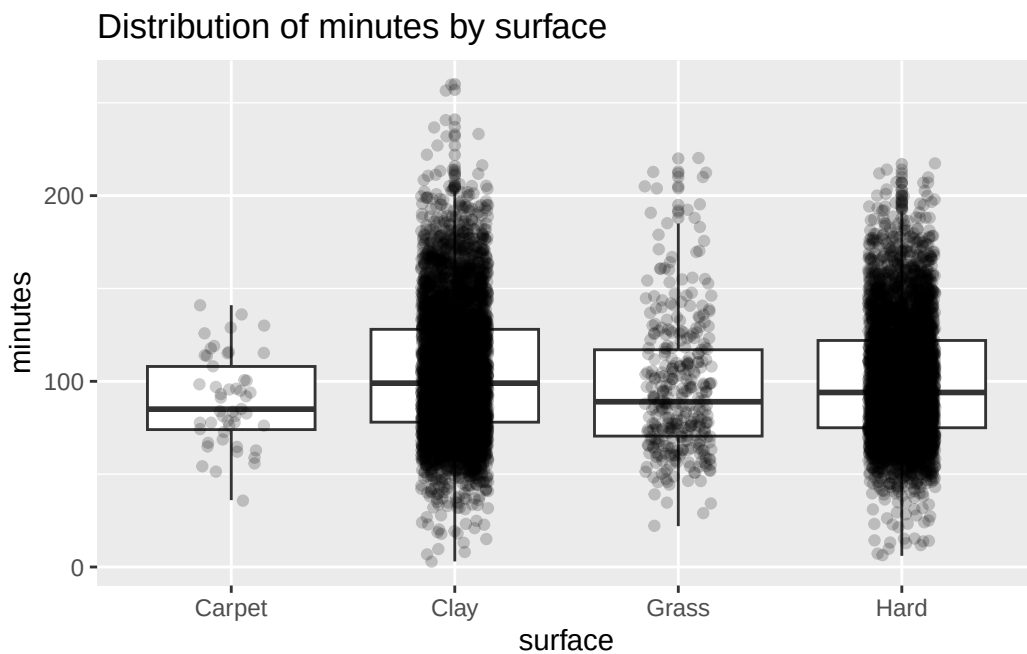
A tibble: 4 x 7

	group_value	n	mean	median	sd	min	max
	<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Carpet	49	89.6	85	24.2	36	141
2	Clay	4907	104.	99	34.8	3	260
3	Grass	343	96.6	89	36.1	22	220
4	Hard	4369	100.	94	32.5	6	217

Kruskal-Wallis test (non-parametric):

```
H0: the distribution of minutes is the same across surface
statistic: 52.14382
df: 3
p-value: 2.790975e-11
```

```
# plot() method  
plot(se_minutes)
```



```
# Example 2 (quick): ace rate by surface (if this column exists)  
se_ace_rate = surface_effect(  
  data      = matches,  
  response  = "aces_per_svpt",  
  group     = "surface",  
  drop_na  = TRUE,  
  run_test  = TRUE)  
  
se_ace_rate
```

```
surface_effect object  
Response: aces_per_svpt  
Grouping variable: surface  
Observations used: 9615  
Number of groups: 4  
Groups: Carpet, Clay, Grass, Hard
```

```
summary(se_ace_rate)
```

Summary for surface_effect

Response: aces_per_svpt

Grouping variable: surface

Group-wise descriptive statistics:

A tibble: 4 x 7

group_value	n	mean	median	sd	min	max
<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 Carpet	49	0.145	0.136	0.0526	0.0360	0.284
2 Clay	4905	0.0407	0.0351	0.0293	0	0.205
3 Grass	342	0.0826	0.0739	0.0397	0	0.228
4 Hard	4319	0.0730	0.0678	0.0385	0	0.5

Kruskal-Wallis test (non-parametric):

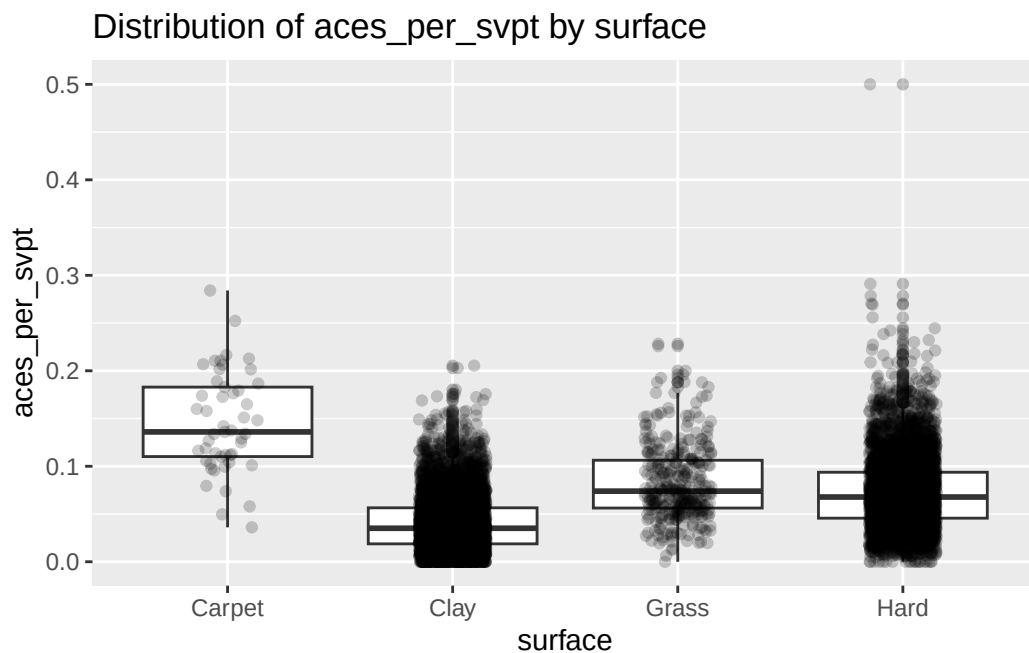
H0: the distribution of aces_per_svpt is the same across surface

statistic: 2122.437

df: 3

p-value: 0

```
plot(se_ace_rate)
```



4.3.1 Interpretation of results

The worked examples demonstrate how the `surface_effect()` function can be used to compare numeric match characteristics across categorical groups using a consistent interface.

For match duration (`minutes`), the Kruskal–Wallis test rejects the null hypothesis that match length distributions are identical across surfaces ($p < 0.001$). This indicates that court surface is meaningfully associated with match duration. The accompanying boxplot shows that Clay matches tend to last longer on average, while Carpet matches are generally shorter, reflecting known differences in surface speed and rally length.

For serve performance (`aces_per_svpt`), the test statistic is even larger, with a p-value effectively equal to zero. This provides overwhelming evidence that ace rates differ by surface. The summary statistics and plot show that faster surfaces such as Carpet and Grass produce higher proportions of aces per serve point, while Clay produces the lowest. This confirms that surface speed amplifies serve effectiveness.

Overall, these examples illustrate that the function is capable of producing informative numerical summaries, formal statistical tests, and clear visualisations within a single, reusable framework.

4.4 Summary of part 3

In this section, I developed a custom R function, `surface_effect()`, to analyse how a numeric outcome varies across groups. The function performs input validation, data cleaning, group-wise descriptive statistics, and an optional Kruskal–Wallis test, returning an object of class “`surface_effect`”.

Custom S3 methods were implemented for `print()`, `summary()`, and `plot()`. Each method serves a distinct purpose: the print method provides a concise overview of the analysis, the summary method presents detailed descriptive statistics and hypothesis test results, and the plot method visualises the distribution of the response variable across groups.

Worked examples using match duration and ace rate demonstrate that the function produces coherent and interpretable results, and that it can be applied flexibly to different numeric outcomes. This section illustrates how user-defined functions and S3 object-oriented programming can be used to build reusable analytical tools in R.

4.5 Conclusion

This project examined differences in match characteristics across playing surfaces in ATP Challenger and qualifying matches using a structured data programming workflow in R. The analysis focused on data cleaning, transformation, summary statistics, and visual exploration

to assess whether surface type is associated with observable variation in match outcomes and related variables.

The results indicate that surface type is linked to differences in several match-level measures, particularly those related to match duration and point structure. While these patterns are consistent with general expectations about faster and slower surfaces, the analysis remains descriptive and does not attempt to establish causal relationships. The findings should therefore be interpreted as associations within the observed dataset rather than definitive surface effects.

From a methodological perspective, the project demonstrates the use of tidy data principles and reproducible reporting through Quarto. The separation of data processing, analysis, and written interpretation allows the results to be checked and reproduced, which is a central requirement of data programming workflows. Visualisations and tables were used to support interpretation rather than replace it.

There are clear limitations to the analysis. The dataset is restricted to a single season and competition level, and no adjustment is made for player specific characteristics or repeated observations. Additionally, the project does not incorporate inferential or predictive modelling, which would be required to formally test hypotheses about surface effects.

In summary, the project provides a structured exploratory analysis of surface related differences in professional tennis matches and illustrates how R can be used to manage, summarise, and communicate data-driven results in a reproducible way.

4.6 References

Firke, S. et al. (2024) ‘janitor: Simple Tools for Examining and Cleaning Dirty Data’. Available at: https://cran.r-project.org/web/packages/janitor/index.html?utm_source=chatgpt.com (Accessed: 3 December 2025).

Nag, U. (2022) Tennis courts: The dimensions, types and all you need to know, Everything you need to know about tennis courts. Available at: <https://www.olympics.com/en/news/tennis-court-markings-dimensions-size-types-variety-surface-hard-grass-clay> (Accessed: 1 December 2025).

Revan, S., Doğan, İ. and Arıkan, Ş. (2021) ‘Analysis of Tennis Competitions on Different Court Surfaces’, *Turkish Journal of Sport and Exercise*, 23(1), pp. 60–66.

Vaverka, F. et al. (2018) ‘Association between serve speed and court surface in tennis’, *International Journal of Performance Analysis in Sport*, 18, pp. 1–11. Available at: <https://doi.org/10.1080/24748668.2018.1467995>.